

AMRITA VISHWA VIDYAPEETHAM

SUBMITTED BY:

NAME	ABDUL HASEEB
ROLL NO	CSE25001
YEAR / SEM / SECTION	2 / 3 / CSE-A
CREDITS	5

SUBMITTED TO:

NAME	Dr.B.RAJ KUMAR
DEPARTMENT	SCHOOL OF COMPUTING
DESIGNATION	ASST. PROFFESOR (SI grade)

S NO:	TITLE	DATE	PAGE
1	WEEK 1: 1a: Recursive countdown from n. 1b: Recursive calculation of compound growth factor. 1c: Recursive search for emo id in list. 1d: Recursive factorial calculation. 1e: Recursive fibonacci series.	23-07-26	5-9
2	WEEK 2: 2a: Write a program to perform linear search on a given input. 2b: Write a Python program to perform binary search on the given input. Verify by taking sorted array & unsorted array as input.	30-07-26	10-12
3	WEEK 3: 3a: write a python program to sort the elements using a bubble sort technique. Take an input elements from the user. 3b: Insertion sort. 3c: selection sort. 3d: Merge sort. 3e: Quick sort.	06-08-26	13-19
4	WEEK 4: 4a: Write a python program to implement a single linked list which would perform the below operations, by taking an input from the user. a. Create a linked list. b. Insert at beginning. c. Insert at end. d. Insert at a specific index. e. Delete by value. f. Delete 1st node. g. Delete last node. h. Count no. of nodes. i. Display. j. Exit. 4b: Doubly linked list 4c: Circular linked list	13-08-26	20-41

5	WEEK 5: 5a: Write a python program to implement stack operations using array which would perform the below operations, by taking an input from the user. a. Push. b. Pop. c. Peek. d. Exit. 5b: Implement stack using linked list	27-08-26	42-46
6	WEEK 6: 6a: Write a python program to implement queue operations using array which would perform the below operations, by taking an input from the user. a. Enqueue. b. Dequeue. c. Peek. d. Exit. 6b: Implement queue using linked list 6c: Implement circular queue using arrays 6d: Implement circular queue using linked list	03-09-26	47-59
7	WEEK 7: 7a: Write a python program to implement binary tree using an array which would perform the below operations, by taking an input from the user. a. Insertion b. Deletion c. Searching d. Traversal 7b: Implement binary tree using linked list	24-09-26	60-65
8	WEEK 8: 8a: Write a Python program to implement a Binary Search Tree (BST) which would perform the below operations, by taking an input from the user. a. Insertion b. Deletion c. Searching d. Traversal	08-10-26	66-74

	8b: Implement a Binary Search Tree using a linked list.		
9	WEEK 9: 9a: Write a Python program to implement a AVL Tree which would perform the following operations by taking input from the user. a. Insertion b. Deletion c. Searching d. Traversal 9b: Implement a AVL Tree using a linked structure and display its elements using traversal.	22-10-26	75-84
10	WEEK 10: 10a: Write a Python program to implement Hashing using Separate Chaining which would perform the following operations by taking input from the user. a. Insertion b. Searching c. Displaying 10b: Write a Python program to implement Hashing using Linear Probing. 10c: Write a Python program to implement Hashing using Quadratic Probing	29-10-26	85-92

LAB 1

1A)

AIM: A rocket launch system displays a countdown. Use no loops. Write a Python program to display the countdown from n to 1, followed by “Launch”.

PROGRAM:

```
def countdown(n):
    if n==0:
        print("Launch")
        return
    print(n)
    countdown(n-1)
n=int(input("Enter n: "))
countdown(n)
```

OUTPUT:

```
======= RESTART: C:/Users/abdul/OneDrive/Documents/dsapy_lab/lab_1/1a.py =======
Enter n: 5
5
4
3
2
1
Launch
```

EXPLANATION:

- Defines a recursive countdown() function that prints numbers in decreasing order.
- Prints the current value of n before making the recursive call.
- When n becomes 0, the recursion stops.
- Otherwise, the function calls itself with n-1.
- Takes the starting number as input and displays the countdown.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Countdown does not stop	Check the condition when n reaches 0
2	Program stops for a very large number	Use a loop instead of recursion
3	Negative numbers are printed unexpectedly	Allow only non negative starting values

1B)

AIM: Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate p^n (power), where p = principal growth factor and n = number of years.

PROGRAM:

```
def power(p,n):
    if n==0:
        return 1
    return p*power(p,n-1)
p=int(input("Enter p: "))
n=int(input("Enter n: "))
print(power(p,n))
```

OUTPUT:

```
======= RESTART: C:/Users/abdul/OneDrive/Documents/dsapy_lab/lab_1/1b.py =======
Enter p: 2
Enter n: 5
32
```

EXPLANATION:

- Defines a recursive power() function to calculate \(p^n\).
- Returns 1 when n <= 0, which is the base case.
- Otherwise, multiplies p by power(p, n-1) repeatedly.
- Takes the base (p) and exponent (n) as input.
- Calls the function and prints the calculated power.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Incorrect result for negative exponent	Add separate handling for negative exponents
2	Program becomes slow for large exponent	Use an efficient power calculation method
3	Wrong result for exponent zero	Make sure the base case returns 1

1C)

AIM: An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

PROGRAM:

```
def search(arr,id,n):
    if n==len(arr):
        return -1
    if arr[n]==id:
        return n
    return search(arr,id,n+1)
arr=[101,102,103,104,105]
id=int(input("Enter employee ID: "))
result=search(arr,id,0)
if result==-1:
    print("Employee ID not found")
else:
    print("Employee ID found at index",result)
```

OUTPUT:

```
======= RESTART: C:/Users/abdul/OneDrive/Documents/dsapy_lab/lab_1/1c.py =======
Enter employee ID: 101
Employee ID found at index 0
```

EXPLANATION:

- Defines a recursive search() function to find the target element in the list.
- Checks each element starting from index 0 using recursion.
- If the target is found, its index is returned.
- If the end of the list is reached, -1 is returned.
- Takes the target value as input and displays the search result.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Target element is not found	Check whether the target exists in the list

2	Recursion stops before checking all elements	Verify the index increment in the recursive call
3	Program stops for a large list	Use an iterative search method

1D)

AIM: In a logistics company, the number of possible ways to arrange n different parcels is calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

PROGRAM:

```
def factorial(n):
    if n==0:
        return 1
    return n*factorial(n-1)
n=int(input("Enter number: "))
print("Factorial =",factorial(n))
```

OUTPUT:

```
======= RESTART: C:/Users/abdul/OneDrive/Documents/dsapy_lab/lab_1/1d.py =======
Enter number: 5
Factorial = 120
```

EXPLANATION:

- Defines a recursive factorial() function to calculate the factorial of a number.
- Returns 1 when n <= 0, which also handles 0!.
- Otherwise, multiplies n by factorial(n-1) until reaching the base case.
- Takes a number as input from the user.
- Calls the function and prints the factorial result.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Incorrect factorial for negative input	Restrict the input to non negative numbers
2	Program stops for a large number	Use an iterative approach

3	Wrong factorial result	Check the recursive multiplication and base condition

1E)

AIM: Write a recursive Python program to print the first n terms of the Fibonacci series.

PROGRAM:

```
def fibonacci(n):
    if n<=1:
        return n
    return fibonacci(n-1)+fibonacci(n-2)
n=int(input("Enter n: "))
for i in range(n):
    print(fibonacci(i),end=" ")
```

OUTPUT:

```
======= RESTART: C:/Users/abdul/OneDrive/Documents/dsapy_lab/lab_1/1e.py ======
Enter n: 5
0 1 1 2 3
```

EXPLANATION:

- Defines a recursive fibonacci() function to calculate Fibonacci numbers.
- Returns 0 for n <= 0 and 1 for n == 1.
- For other values, it recursively adds the previous two Fibonacci numbers.
- Takes the number of terms from the user.
- Uses a loop to generate and print each Fibonacci term.
- The sequence starts as 0, 1, 1, 2, 3, 5, ...

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Program runs very slowly	Use an iterative or optimized approach
2	Incorrect sequence is displayed	Check the base conditions for 0 and 1
3	Program stops for large input	Avoid excessive recursive calls

LAB 2

2A)

AIM: Write a program to perform linear search on a given input.

PROGRAM:

```
def linear_search(arr,x):
    for i in range(len(arr)):
        if arr[i]==x:
            return i
    return -1
arr=[]
n=int(input("Enter number of elements: "))
for i in range(n):
    arr.append(int(input("Enter element: ")))
x=int(input("Enter element to search: "))
result=linear_search(arr,x)
if result==-1:
    print("Element not found")
else:
    print("Element found at index",result)
```

OUTPUT:

```
======= RESTART: C:/Users/abdul/OneDrive/Documents/dsapy_lab/lab_2/2a.py =======
Enter number of elements: 5
Enter element: 12
Enter element: 45
Enter element: 63
Enter element: 4
Enter element: 7
Enter element to search: 7
Element found at index 4
```

EXPLANATION:

- Takes the number of elements and stores them in a list.
- Takes the target element to be searched.
- linearSearch() checks each element one by one.
- If the target is found, its index is returned.
- If not found, -1 is returned.
- The result is displayed using print().

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Element is not found	Check whether the required element is present in the list
2	Wrong index is displayed	Check the list indexing starting from 0
3	Search stops unexpectedly	Verify the range used in the for loop

2B)

AIM: Write a Python program to perform binary search on the given input. Verify by taking sorted array & unsorted array as input.

PROGRAM:

```
def binary_search(arr,x):
    low=0
    high=len(arr)-1
    while low<=high:
        mid=(low+high)//2
        if arr[mid]==x:
            return mid
        elif arr[mid]<x:
            low=mid+1
        else:
            high=mid-1
    return -1
arr=[]
n=int(input("Enter number of elements: "))
for i in range(n):
    arr.append(int(input("Enter element: ")))
arr.sort()
x=int(input("Enter element to search: "))
result=binary_search(arr,x)
if result==-1:
```

```
    print("Element not found")
else:
    print("Element found at index",result)
```

OUTPUT:

```
======= RESTART: C:/Users/abdul/OneDrive/Documents/dsapy_lab/lab_2/2b.py =======
Enter number of elements: 5
Enter element: 12
Enter element: 2
Enter element: 45
Enter element: 6
Enter element: 8
Enter element to search: 6
Element found at index 1
```

EXPLANATION:

- Takes the number of elements and stores them in a list.
- Sorts the list using a.sort() because binary search requires sorted data.
- Calculates the middle element and compares it with the target.
- If the target is smaller or larger, searches only the appropriate half.
- Returns the index if found; otherwise returns -1.
- Displays whether the element is found or not.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Wrong element is returned	Make sure the list is sorted before searching
2	Search gives incorrect index	Check the calculation of the middle index
3	Element is not found	Verify the low and high boundaries

LAB 3

3A)

AIM: Write a python programme to sort the elements using a bubble sort technique. Take an input elements from the users.

PROGRAM:

```
def bubble_sort(a, n):
    for i in range(n):
        for j in range(n - i - 1):
            if a[j] > a[j + 1]:
                a[j], a[j + 1] = a[j + 1], a[j]
    return a
n = int(input("Enter number of elements: "))
a = []
for i in range(n):
    x = int(input("Enter element: "))
    a.append(x)
print("Sorted array:", bubble_sort(a, n))
```

OUTPUT:

```
======= RESTART: C:\Users\abdul\OneDrive\Documents\dsapy_lab\lab_3\3a.py =======
Enter number of elements: 4
Enter element: 23
Enter element: 22
Enter element: 21
Enter element: 45
Sorted array: [21, 22, 23, 45]
```

EXPLANATION:

Defines the bubble_sort function and finds the number of elements in the list.
Compares adjacent elements using two nested loops.
Swaps the elements if they are in the wrong order.
Repeats the process until all elements are arranged in ascending order.
Takes the elements as input from the user and stores them in a list.
Calls bubble_sort and displays the sorted array.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	IndexErrorwhile comparing elements	Check the loop range and array size
2	Elements are not sorted correctly	Check the swapping condition
3	Incorrect number of elements processed	Make sure the entered count matches the elements

3B)

AIM: Write a python programme to sort the elements using a insertion sort technique. Take an input elements from the users.

PROGRAM:

```
def insertion_sort(a, n):
    for i in range(1, n):
        key = a[i]
        for j in range(i - 1, -1, -1):
            if a[j] > key:
                a[j + 1] = a[j]
            else:
                break
        a[j + 1] = key
    return a
n = int(input("Enter number of elements: "))
a = []
for i in range(n):
    x = int(input("Enter element: "))
    a.append(x)
print("Sorted array:", insertion_sort(a, n))
```

OUTPUT:

```
======= RESTART: C:\Users\abdul\OneDrive\Documents\dsapy_lab\lab_3\3c.py =======
Enter number of elements: 4
Enter element: 23
Enter element: 21
Enter element: 14
Enter element: 24
Sorted array: [23, 14, 21, 24]
```

EXPLANATION:

Defines the insertion_sort function and starts from the second element of the list.
Stores the current element in key and compares it with the previous elements.
Moves larger elements one position to the right.
Places key in its correct position in the sorted portion of the list.
Displays the original and sorted arrays.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Elements are not sorted correctly	Check the position where key is inserted
2	IndexErrorwhile shifting elements	Check the condition j >= 0
3	Someelementsare overwritten	Place key after completing the shifting process

3C)

AIM: Write a python programme to sort the elements using a selection sort technique. Take an input elements from the users.

PROGRAM:

```
def selection_sort(a, n):
    for i in range(n):
        min_index = i
        for j in range(i + 1, n):
            if a[j] < a[min_index]:
                min_index = j
        a[i], a[min_index] = a[min_index], a[i]
    return a
n = int(input("Enter number of elements: "))
a = []
for i in range(n):
```

```
    x = int(input("Enter element: "))
    a.append(x)
print("Sorted array:", selection_sort(a, n))
```

OUTPUT:

```
======= RESTART: C:\Users\abdul\OneDrive\Documents\dsapy_lab\lab_3\3b.py =======
Enter number of elements: 4
Enter element: 32
Enter element: 12
Enter element: 45
Enter element: 76
Sorted array: [12, 32, 45, 76]
>
```

EXPLANATION:

Defines the selection_sort function and assumes the first unsorted element as the minimum.
Compares it with the remaining elements to find the smallest element.
Stores the position of the smallest element in min_index.
Swaps the smallest element with the element at the current position.
Repeats the process until the entire list is sorted.
Takes input elements and displays the sorted array.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Incorrect sorting of elements	Perform the swap after finding the minimum element
2	Elements are swapped multiple times	Move the swap statement outside the inner loop
3	Minimum element is selected incorrectly	Check the min_index update condition

3D)

AIM: Write a python programme to sort the elements using a merge sort technique. Take an input elements from the users.

PROGRAM:

```
def merge_sort(a):
    if len(a) <= 1:
```

```
        return a
    mid = len(a)
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])
    result = []
    i = 0
    j = 0
    for k in range(len(left) + len(right)):
        if i < len(left) and j < len(right):
            if left[i] <= right[j]:
                result.append(left[i])
                i += 1
            else:
                result.append(right[j])
                j += 1
        elif i < len(left):
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    return result
n = int(input("Enter number of elements: "))
a = []
for i in range(n):
    x = int(input("Enter element: "))
    a.append(x)
a = merge_sort(a)
print("Sorted array:", a)
```

OUTPUT:

```
======= RESTART: C:\Users\abdul\OneDrive\Documents\dsapy_lab\lab_3\3b.py =======
Enter number of elements: 4
Enter element: 32
Enter element: 12
Enter element: 45
Enter element: 76
Sorted array: [12, 32, 45, 76]
>
```

EXPLANATION:

Defines the merge_sort function and divides the array into two halves.
Recursively divides each half until single elements remain.
Compares elements from the left and right halves.

Merges the elements back into the array in ascending order.
Continues merging until the complete array is sorted
Takes input elements and displays the original and sorted arrays.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Elements are not merged correctly	Check the comparison between left and right arrays
2	Some elements are missing after merging	Check the remaining element loops
3	Incorrect sorted output	Verify the values of i, j and k during merging

3E)

AIM: Write a python programme to sort the elements using a quick sort technique. Take an input elements from the users.

PROGRAM:

```
def quick_sort(a):
    if len(a) <= 1:
        return a
    pivot = a[-1]
    left = []
    right = []
    for i in range(len(a) - 1):
        if a[i] <= pivot:
            left.append(a[i])
        else:
            right.append(a[i])
    return quick_sort(left) + [pivot] + quick_sort(right)
n = int(input("Enter number of elements: "))
a = []
for i in range(n):
    x = int(input("Enter element: "))
    a.append(x)
a = quick_sort(a)
print("Sorted array:", a)
```

OUTPUT:

```
======= RESTART: C:\Users\abdul\OneDrive\Documents\dsapy_lab\lab_3\3e.py =======
Enter number of elements: 4
Enter element: 21
Enter element: 32
Enter element: 54
Enter element: 2
Sorted array: [2, 21, 32, 54]
```

EXPLANATION:

Defines a recursive Quick Sort function and selects the first element as the pivot.
Uses two pointers, i and j, to compare and arrange elements around the pivot.
Swaps elements when they are on the wrong side of the pivot.
Places the pivot in its correct position after partitioning.
Recursively sorts the elements before and after the pivot.
Takes the elements as input and performs the sorting operation.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	UnboundLocalError for i or j	Keep the partitioning code inside the if block
2	RecursionError for large input	Check the recursive calls and pivot position
3	Sorted array is displayed as None	Print the list after calling quicksort

\

LAB 4

4A)

AIM: Write a python programme to implement single linked list. Take an input elements from the users .

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class SLL:
    def __init__(self):
        self.head = None

    # 1. Create a linked list
    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input("Enter data: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
            else:
                temp = self.head

                while temp.next is not None:
                    temp = temp.next

                temp.next = new_node

        print("Linked List created successfully")

    # 2. Insert at Beginning
    def insert_beginning(self):
        data = int(input("Enter data: "))

        new_node = Node(data)
```

```
        new_node.next = self.head
        self.head = new_node

        print(f"Inserted {data} at the beginning.")

    # 3. Insert at End
    def insert_end(self):
        data = int(input("Enter data: "))

        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            return

        temp = self.head

        while temp.next is not None:
            temp = temp.next

        temp.next = new_node

        print(f"Inserted {data} at the end.")

    # 4. Insert at Index
    def insert_at_index(self):
        data = int(input("Enter data: "))
        index = int(input("Enter index: "))

        if index < 0:
            print("Invalid index")
            return

        if index == 0:
            new_node = Node(data)
            new_node.next = self.head
            self.head = new_node
            return

        temp = self.head

        for i in range(index - 1):
            if temp is None:
```

```
            print("Index out of range")
            return
        temp = temp.next

    if temp is None:
        print("Index out of range")
        return

    new_node = Node(data)
    new_node.next = temp.next
    temp.next = new_node

    print(f"Inserted {data} at index {index}.")

# 5. Delete from Beginning
def delete_beginning(self):
    if self.head is None:
        print("List is empty")
        return

    self.head = self.head.next
    print("Deleted beginning node.")

# 6. Delete from End
def delete_end(self):
    if self.head is None:
        print("List is empty")
        return

    if self.head.next is None:
        self.head = None
        return

    temp = self.head

    while temp.next.next is not None:
        temp = temp.next

    temp.next = None

    print("Deleted last node.")

# 7. Delete from Index
```

```
def delete_at_index(self):
    index = int(input("Enter index: "))

    if self.head is None:
        print("List is empty")
        return

    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        self.head = self.head.next
        return

    temp = self.head

    for i in range(index - 1):
        if temp.next is None:
            print("Index out of range")
            return
        temp = temp.next

    if temp.next is None:
        print("Index out of range")
        return

    temp.next = temp.next.next

    print("Deleted node value.")

# 8. Count Nodes
def count_nodes(self):
    count = 0
    temp = self.head

    while temp is not None:
        count += 1
        temp = temp.next

    print("Number of nodes:", count)

# 9. Display
```

```
    def display(self):
        if self.head is None:
            print("List is empty")
            return

        temp = self.head

        while temp is not None:
            print(temp.data, end=" -> ")
            temp = temp.next

        print("None")


# Main Program
sll = SLL()

while True:
    print("\n----- SINGLY LINKED LIST -----")
    print("1. Create Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete from Beginning")
    print("6. Delete from End")
    print("7. Delete from Index")
    print("8. Count number of nodes")
    print("9. Display")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        sll.create()

    elif choice == 2:
        sll.insert_beginning()

    elif choice == 3:
        sll.insert_end()

    elif choice == 4:
        sll.insert_at_index()
```

```
    elif choice == 5:
        sll.delete_beginning()

    elif choice == 6:
        sll.delete_end()

    elif choice == 7:
        sll.delete_at_index()

    elif choice == 8:
        sll.count_nodes()

    elif choice == 9:
        sll.display()

    elif choice == 10:
        print("Exiting from the program...")
        break

    else:
        print("Invalid choice")
```

OUTPUT:

```
======= RESTART: C:\Users\abdul\OneDrive\Documents\dsapy_lab\lab_4\4a.py =======

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count number of nodes
9. Display
10. Exit
Enter your choice: 1
Enter number of nodes: 4
Enter data: 12
Enter data: 24
Enter data: 45
Enter data: 6
Linked List created successfully

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count number of nodes
9. Display
10. Exit
Enter your choice:
```

EXPLANATION:

Defines a node containing data and a pointer to the next node.
Creates a singly linked list by dynamically allocating memory for each node.
Stores the address of the next node in the next pointer.
The last node points to NULL, indicating the end of the list.
Performs operations such as insertion, deletion, and traversal.
Traverses the list from the first node until NULL is reached.
Takes the required elements as input and performs the linked list operations.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	New node is not inserted correctly	Update the next pointer of the previous node
2	List traversal stops incorrectly	Make sure the last node points to NULL
3	First node is lost after insertion	Update the head pointer correctly

4B)

AIM: Write a python programme to implement double linked list. Take an input elements from the users.

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None


class DLL:
    def __init__(self):
        self.head = None

    # 1. Create Linked List
    def create(self):
        n = int(input("Enter number of nodes: "))
```

```
    for i in range(n):
        data = int(input("Enter data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
            temp = self.head

            while temp.next is not None:
                temp = temp.next

            temp.next = new_node
            new_node.prev = temp

    print("Linked List created successfully")

# 2. Insert at Beginning
def insert_beginning(self):
    data = int(input("Enter data: "))

    new_node = Node(data)

    new_node.next = self.head

    if self.head is not None:
        self.head.prev = new_node

    self.head = new_node

    print(f"Inserted {data} at the beginning.")

# 3. Insert at End
def insert_end(self):
    data = int(input("Enter data: "))

    new_node = Node(data)

    if self.head is None:
        self.head = new_node
        print(f"Inserted {data} at the end.")
        return
```

```
    temp = self.head

    while temp.next is not None:
        temp = temp.next

    temp.next = new_node
    new_node.prev = temp

    print(f"Inserted {data} at the end.")

# 4. Insert at Index
def insert_at_index(self):
    data = int(input("Enter data: "))
    index = int(input("Enter index: "))

    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        new_node = Node(data)
        new_node.next = self.head

        if self.head is not None:
            self.head.prev = new_node

        self.head = new_node
        return

    temp = self.head

    for i in range(index - 1):
        if temp is None:
            print("Index out of range")
            return

        temp = temp.next

    if temp is None:
        print("Index out of range")
        return
```

```
        new_node = Node(data)

        new_node.next = temp.next
        new_node.prev = temp

        if temp.next is not None:
            temp.next.prev = new_node

        temp.next = new_node

        print(f"Inserted {data} at index {index}.")

    # 5. Delete from Beginning
    def delete_beginning(self):
        if self.head is None:
            print("List is empty")
            return

        self.head = self.head.next

        if self.head is not None:
            self.head.prev = None

        print("Deleted beginning node.")

    # 6. Delete from End
    def delete_end(self):
        if self.head is None:
            print("List is empty")
            return

        if self.head.next is None:
            self.head = None
            print("Deleted last node.")
            return

        temp = self.head

        while temp.next is not None:
            temp = temp.next

        temp.prev.next = None
```

```
    print("Deleted last node.")

# 7. Delete from Index
def delete_at_index(self):
    index = int(input("Enter index: "))

    if self.head is None:
        print("List is empty")
        return

    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        self.head = self.head.next

        if self.head is not None:
            self.head.prev = None

        print("Deleted node value.")
        return

    temp = self.head

    for i in range(index):
        if temp is None:
            print("Index out of range")
            return

        temp = temp.next

    if temp is None:
        print("Index out of range")
        return

    if temp.next is not None:
        temp.next.prev = temp.prev

    if temp.prev is not None:
        temp.prev.next = temp.next

    print("Deleted node value.")
```

```
    # 8. Count Nodes
    def count_nodes(self):
        count = 0
        temp = self.head

        while temp is not None:
            count += 1
            temp = temp.next

        print("Number of nodes:", count)

    # 9. Display
    def display(self):
        if self.head is None:
            print("List is empty")
            return

        temp = self.head

        while temp is not None:
            print(temp.data, end=" <-> ")
            temp = temp.next

        print("None")


# Main Program
dll = DLL()

while True:
    print("\n----- DOUBLY LINKED LIST -----")
    print("1. Create Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete from Beginning")
    print("6. Delete from End")
    print("7. Delete from Index")
    print("8. Count number of nodes")
    print("9. Display")
    print("10. Exit")
```

```
choice = int(input("Enter your choice: "))

if choice == 1:
    dll.create()

elif choice == 2:
    dll.insert_beginning()

elif choice == 3:
    dll.insert_end()

elif choice == 4:
    dll.insert_at_index()

elif choice == 5:
    dll.delete_beginning()

elif choice == 6:
    dll.delete_end()

elif choice == 7:
    dll.delete_at_index()

elif choice == 8:
    dll.count_nodes()

elif choice == 9:
    dll.display()

elif choice == 10:
    print("Exiting from the program...")
    break

else:
    print("Invalid choice")
```

OUTPUT:

```
======= RESTART: C:\Users\abdul\OneDrive\Documents\dsapy_lab\lab_4\4b.py =======

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count number of nodes
9. Display
10. Exit
Enter your choice: 1
Enter number of nodes: 4
Enter data: 23
Enter data: 54
Enter data: 69
Enter data: 54
Linked List created successfully
```

EXPLANATION:

Defines a node containing data, a pointer to the previous node, pointer to next.
Creates a doubly linked list by dynamically allocating memory for each node.
Maintains both prev and next pointers for every node.
The first node has prev as NULL.
The last node has next as NULL.
Performs operations such as insertion, deletion, forward traversal and back.
Updates both prev and next pointers whenever a node is inserted or deleted.
Takes the required elements as input and performs the linked list operations.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Segmentation fault while accessing prev or next	Check whether the node is NULL before accessing it
2	Forward traversal works but backward traversal fails	Update the prev pointer correctly
3	Node is inserted but links are incorrect	Update both prev and next pointers

4C)

AIM: Write a python programme to implement circular linked list. Take an input elements from the users.

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CLL:
    def __init__(self):
        self.head = None

    # 1. Create Linked List
    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input("Enter data: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
                new_node.next = self.head
            else:
                temp = self.head

                while temp.next != self.head:
                    temp = temp.next

                temp.next = new_node
                new_node.next = self.head

        print("Linked List created successfully")

    # 2. Insert at Beginning
    def insert_beginning(self):
        data = int(input("Enter data: "))
```

```
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            new_node.next = self.head
            print(f"Inserted {data} at the beginning.")
            return

        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        new_node.next = self.head
        self.head = new_node
        temp.next = self.head

        print(f"Inserted {data} at the beginning.")

    # 3. Insert at End
    def insert_end(self):
        data = int(input("Enter data: "))

        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            new_node.next = self.head
            print(f"Inserted {data} at the end.")
            return

        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        temp.next = new_node
        new_node.next = self.head

        print(f"Inserted {data} at the end.")

    # 4. Insert at Index
    def insert_at_index(self):
```

```
data = int(input("Enter data: "))
index = int(input("Enter index: "))

if index < 0:
    print("Invalid index")
    return

if index == 0:
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
        new_node.next = self.head
    else:
        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        new_node.next = self.head
        self.head = new_node
        temp.next = self.head

    print(f"Inserted {data} at index {index}.")
    return

if self.head is None:
    print("Index out of range")
    return

temp = self.head

for i in range(index - 1):
    temp = temp.next

    if temp == self.head:
        print("Index out of range")
        return

new_node = Node(data)

new_node.next = temp.next
temp.next = new_node
```

```
    print(f"Inserted {data} at index {index}.")

# 5. Delete from Beginning
def delete_beginning(self):
    if self.head is None:
        print("List is empty")
        return

    if self.head.next == self.head:
        self.head = None
        print("Deleted beginning node.")
        return

    temp = self.head

    while temp.next != self.head:
        temp = temp.next

    self.head = self.head.next
    temp.next = self.head

    print("Deleted beginning node.")

# 6. Delete from End
def delete_end(self):
    if self.head is None:
        print("List is empty")
        return

    if self.head.next == self.head:
        self.head = None
        print("Deleted last node.")
        return

    temp = self.head

    while temp.next.next != self.head:
        temp = temp.next

    temp.next = self.head

    print("Deleted last node.")
```

```
# 7. Delete from Index
def delete_at_index(self):
    index = int(input("Enter index: "))

    if self.head is None:
        print("List is empty")
        return

    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        self.delete_beginning()
        return

    temp = self.head

    for i in range(index - 1):
        temp = temp.next

        if temp == self.head:
            print("Index out of range")
            return

    if temp.next == self.head:
        print("Index out of range")
        return

    temp.next = temp.next.next

    print("Deleted node value.")

# 8. Count Nodes
def count_nodes(self):
    if self.head is None:
        print("Number of nodes: 0")
        return

    count = 0
    temp = self.head
```

```
        while True:
            count += 1
            temp = temp.next

            if temp == self.head:
                break

        print("Number of nodes:", count)

    # 9. Display
    def display(self):
        if self.head is None:
            print("List is empty")
            return

        temp = self.head

        while True:
            print(temp.data, end=" -> ")
            temp = temp.next

            if temp == self.head:
                break

        print("(HEAD)")


# Main Program
cll = CLL()

while True:
    print("\n----- CIRCULAR LINKED LIST -----")
    print("1. Create Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete from Beginning")
    print("6. Delete from End")
    print("7. Delete from Index")
    print("8. Count number of nodes")
    print("9. Display")
    print("10. Exit")
```

```
choice = int(input("Enter your choice: "))

if choice == 1:
    cll.create()

elif choice == 2:
    cll.insert_beginning()

elif choice == 3:
    cll.insert_end()

elif choice == 4:
    cll.insert_at_index()

elif choice == 5:
    cll.delete_beginning()

elif choice == 6:
    cll.delete_end()

elif choice == 7:
    cll.delete_at_index()

elif choice == 8:
    cll.count_nodes()

elif choice == 9:
    cll.display()

elif choice == 10:
    print("Exiting from the program...")
    break

else:
    print("Invalid choice")
```

OUTPUT:

```
Enter your choice:
======= RESTART: C:\Users\abdul\OneDrive\Documents\dsapy_lab\lab_4\4c.py =======

----- CIRCULAR LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count number of nodes
9. Display
10. Exit
Enter your choice: 1
Enter number of nodes: 4
Enter data: 12
Enter data: 56
Enter data: 79
Enter data: 56
Linked List created successfully
```

EXPLANATION:

Defines a node containing data and a pointer to the next node.
Creates a circular linked list by dynamically allocating memory for each node.
The last node points back to the first node instead of NULL.
Traverses the list starting from the first node.
Stops traversal when the current node reaches the starting node again.
Performs operations such as insertion, deletion, and traversal.
Maintains the circular connection after every insertion or deletion
Takes the required elements as input and performs the circular linked list.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Infinite loop during traversal	Stop when the current node becomes the starting node again
2	Last node does not point to the first node	Set the last node's next pointer to head
3	Segmentation fault while accessing a node	Check whether the list is empty before accessing nodes

LAB 5

5A)

AIM: Write a python programme to implement stack operations using arrays. Take an input elements from the users.

PROGRAM:

```
class StackEx:
    def __init__(self, size):
        self.size = size
        self.stack = [None] * size
        self.top = -1

    def push(self, item):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.top += 1
            self.stack[self.top] = item
            print(item, "pushed into the stack")

    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            print(item, "popped from the stack")

    def peek(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Top element:", self.stack[self.top])

    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("The elements of the stack are:")
            for i in range(self.top, -1, -1):
                print(self.stack[i])
```

```
size = int(input("Enter the size of the stack: "))
s = StackEx(size)

while True:
    print("\n----- STACK MENU -----")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to push: "))
        s.push(item)
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        print("Program terminated.")
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
======= RESTART: C:\Users\abdul\OneDrive\Documents\dsapy_lab\lab_5\5a.py =======
Enter the size of the stack: 4

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to push: 12
12 pushed into the stack

----- STACK MENU -----
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice:
```

EXPLANATION:

Defines the stack array and initializes the top variable to -1. Checks whether the stack is full before performing the push operation. Adds elements to the top of the stack using the push operation. Checks whether the stack is empty before performing the pop operation. Removes the top element using the pop operation and decreases top. Displays the top element using the peek operation.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Stack Overflow	Check whether top == MAX-1 before pushing
2	Stack Underflow	Check whether top == -1 before popping
3	Incorrect top position	Check the increment and decrement of top

5B)

AIM: Write a python programme to implement stack operations using linked list. Take an input elements from the users.

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Stack:
    def __init__(self):
        self.top = None

    def push(self, data):
        new = Node(data)
        new.next = self.top
        self.top = new
        print(data, "pushed into stack")

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
```

```
        else:
            temp = self.top
            print(temp.data, "popped from stack")
            self.top = self.top.next

    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            temp = self.top
            print("Stack elements:")
            while temp is not None:
                print(temp.data)
                temp = temp.next

s = Stack()

while True:
    print("\n--- STACK USING LINKED LIST ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter the element: "))
        s.push(data)
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        print("Program terminated")
        break
    else:
```

print("Invalid choice")

OUTPUT:

```
======= RESTART: C:\Users\abdul\OneDrive\Documents\dsapy_lab\lab_5\5b.py =======

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element: 25
25 pushed into stack

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice:
```

EXPLANATION:

Defines the node structure containing data and a pointer to the next node. Creates a new node and inserts it at the top during the push operation. Checks whether the stack is empty before performing the pop operation. Removes the top node and updates the top pointer. Frees the deleted node and displays the stack elements.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	IndexErrorwhile comparing elements	Check the loop range and array size
2	Elements are not sorted correctly	Check the swapping condition
3	Incorrect number of elements processed	Make sure the entered count matches the elements

LAB 6

6A)

AIM: Write a python programme to implement queue operations using arrays. Take an input elements from the users.

PROGRAM:

```
class QueueEx:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, item):
        if self.rear == self.size - 1:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0

            self.rear += 1
            self.queue[self.rear] = item
            print(item, "enqueued into the queue")

    def dequeue(self):
        if self.front == -1 or self.front > self.rear:
            print("Queue Underflow")
        else:
            item = self.queue[self.front]
            self.queue[self.front] = None
            self.front += 1

            if self.front > self.rear:
                self.front = -1
                self.rear = -1

            print(item, "dequeued from the queue")

    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])
```

```
    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            for i in range(self.front, self.rear + 1):
                print(self.queue[i])


size = int(input("Enter the size of the queue: "))
q = QueueEx(size)

while True:
    print("\n----- QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Program terminated.")
        break

    else:
        print("Invalid choice")
```

OUTPUT:

```
======= RESTART: C:\Users\abdul\OneDrive\Documents\dsapy_lab\lab_6\6a.py =======
Enter the size of the queue: 5

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 45
45 enqueued into the queue

----- QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice:
```

EXPLANATION:

Defines the queue array and initializes the front and rear variables. Checks whether the queue is full before performing the enqueue operation. Adds elements at the rear of the queue using the enqueue operation. Checks whether the queue is empty before performing the dequeue operation. Removes elements from the front of the queue using the dequeue operation. Displays the elements of the queue from front to rear.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Queue Overflow	Check whether rear == MAX-1 before enqueue
2	Queue Underflow	Check whether front == -1 or front > rear before dequeue
3	Incorrect front or rear position	Check the increment and update of front and rear

6B)

AIM: Write a python programme to implement queue operations using linked list. Take an input elements from the users.

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class QueueEx:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, item):
        new_node = Node(item)

        if self.front is None:
            self.front = new_node
            self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node

        print(item, "enqueued into the queue")

    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
        else:
            item = self.front.data
            self.front = self.front.next

            if self.front is None:
                self.rear = None

            print(item, "dequeued from the queue")

    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element:", self.front.data)
```

```
    def display(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")

            temp = self.front

            while temp is not None:
                print(temp.data)
                temp = temp.next


q = QueueEx()

while True:
    print("\n----- QUEUE USING LINKED LIST -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Program terminated.")
        break

    else:
        print("Invalid choice")
```

OUTPUT:

```
======= RESTART: C:\Users\abdul\OneDrive\Documents\dsapy_lab\lab_6\6b.py =======

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 1
1 enqueued into the queue

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice:
```

EXPLANATION:

Defines the node structure containing data and a pointer to the next node.
Initializes the front and rear pointers to NULL.
Creates a new node and inserts it at the rear during the enqueue operation.
Checks whether the queue is empty before performing the dequeue operation.
Removes the node from the front and updates the front pointer.
Updates the rear pointer when the last element is removed.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Queue Underflow	Check whether front == NULL before dequeue
2	Memory allocation error	Check whether the new node is successfully allocated
3	Incorrect front or rear pointer	Check the updating of front and rear

6C)

AIM: Write a python programme to implement circular queue operations using arrays. Take an input elements from the users.

PROGRAM:

```
class CircularQueueEx:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

    def enqueue(self, item):
        if (self.rear + 1) % self.size == self.front:
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0
                self.rear = 0
            else:
                self.rear = (self.rear + 1) % self.size

            self.queue[self.rear] = item
            print(item, "enqueued into the circular queue")

    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow")
        else:
            item = self.queue[self.front]
            self.queue[self.front] = None

            if self.front == self.rear:
                self.front = -1
                self.rear = -1
            else:
                self.front = (self.front + 1) % self.size

            print(item, "dequeued from the circular queue")

    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element:", self.queue[self.front])

    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the circular queue are:")
```

```
            i = self.front

            while True:
                print(self.queue[i])

                if i == self.rear:
                    break

                i = (i + 1) % self.size


size = int(input("Enter the size of the circular queue: "))
q = CircularQueueEx(size)

while True:
    print("\n----- CIRCULAR QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

    elif choice == 4:
        q.display()

    elif choice == 5:
        print("Program terminated.")
        break

    else:
        print("Invalid choice")
```

OUTPUT:

```
======= RESTART: C:\Users\abdul\OneDrive\Documents\dsapy_lab\lab_6\6c.py =======
Enter the size of the circular queue: 4

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 12
12 enqueued into the circular queue

----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice:
```

EXPLANATION:

Defines the circular queue array and initializes front and rear. Checks whether the circular queue is full before performing the enqueue operation.
Adds elements at the rear and uses modulo operation to wrap around the array. Checks whether the queue is empty before performing the dequeue operation. Removes elements from the front and updates the position using modulo operation.
Allows the unused spaces at the beginning of the array to be reuse.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	CircularQueue Overflow	Check whether (rear + 1) % MAX == front
2	CircularQueue Underflow	Check whether front == -1
3	Incorrectcircular movement	Use modulo operation correctly

6D)

AIM: Write a python programme to implement circular queue operations using arrays. Take an input elements from the users.

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularQueueEx:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, item):
        new_node = Node(item)

        if self.front is None:
            self.front = new_node
            self.rear = new_node
            self.rear.next = self.front
        else:
            new_node.next = self.front
            self.rear.next = new_node
            self.rear = new_node

        print(item, "enqueued into the circular queue")

    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
        else:
            item = self.front.data

            if self.front == self.rear:
                self.front = None
                self.rear = None
            else:
                self.front = self.front.next
                self.rear.next = self.front

            print(item, "dequeued from the circular queue")

    def peek(self):
```

```
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element:", self.front.data)

    def display(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("The elements of the circular queue are:")

            temp = self.front

            while True:
                print(temp.data)

                if temp == self.rear:
                    break

                temp = temp.next


q = CircularQueueEx()

while True:
    print("\n----- CIRCULAR QUEUE USING LINKED LIST -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)

    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.peek()

    elif choice == 4:
        q.display()
```

```
    elif choice == 5:
        print("Program terminated.")
        break

    else:
        print("Invalid choice")
```

OUTPUT:

```
======= RESTART: C:\Users\abdul\OneDrive\Documents\dsapy_lab\lab_6\6d.py =======

----- CIRCULAR QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to enqueue: 56
56 enqueued into the circular queue

----- CIRCULAR QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice:
```

EXPLANATION:

Defines the node structure containing data and a pointer to the next node. Initializes the front and rear pointers to NULL. Creates a new node and inserts it at the rear of the circular queue. Connects the last node back to the first node to maintain the circular structure. Removes elements from the front during the dequeue operation.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Queue Underflow	Check whether front == NULL before dequeue
2	Memory allocation error	Check whether the new node is successfully allocated

3	Incorrect circular link	Ensure rear->next points to front

LAB 7

7A)

AIM: Write a python program to implement binary tree using an array

PROGRAM:

```
class BinaryTreeArray:
    def __init__(self,size):
        self.tree=[None]*size
        self.size=size

    def set_root(self,data):
        self.tree[0]=data

    def set_left(self,parent_index,data):
        child_index=2*parent_index+1
        if child_index<self.size:
            self.tree[child_index]=data
        else:
            print("Index out of range!")

    def set_right(self,parent_index,data):
        child_index=2*parent_index+2
        if child_index<self.size:
            self.tree[child_index]=data
        else:
            print("Index out of range!")

    def preorder(self,index=0):
        if index>=self.size or self.tree[index] is None:
            return
        print(self.tree[index],end=" ")
        self.preorder(2*index+1)
        self.preorder(2*index+2)

    def inorder(self,index=0):
        if index>=self.size or self.tree[index] is None:
            return
        self.inorder(2*index+1)
        print(self.tree[index],end=" ")
        self.inorder(2*index+2)

    def postorder(self,index=0):
        if index>=self.size or self.tree[index] is None:
            return
```

```
        self.postorder(2*index+1)
        self.postorder(2*index+2)
        print(self.tree[index],end=" ")

    def level_order(self):
        for value in self.tree:
            if value is not None:
                print(value,end=" ")

    def display(self):
        print("\nArray Representation:")
        for i in range(self.size):
            print(f"Index {i}: {self.tree[i]}")

tree=BinaryTreeArray(7)
tree.set_root('A')
tree.set_left(0,'B')
tree.set_right(0,'C')
tree.set_left(1,'D')
tree.set_right(1,'E')
tree.set_right(2,'F')
tree.display()
print("\nPreorder: ",end="")
tree.preorder()
print("\nInorder: ",end="")
tree.inorder()
print("\nPostorder: ",end="")
tree.postorder()
print("\nLevel-order: ",end="")
tree.level_order()
```

OUTPUT:

```
======= RESTART: C:\Users\abdul\OneDrive\Documents\dsapy_lab\lab_7\7a.py =======

Array Representation:
Index 0: A
Index 1: B
Index 2: C
Index 3: D
Index 4: E
Index 5: None
Index 6: F

Preorder: A B D E C F
Inorder: D B E A C F
Postorder: D E B F C A
Level-order: A B C D E F
```

EXPLANATION:

Defines a binary tree using an array to store the elements.
The root node is stored at index 0.
For a node at index i, the left child is at index 2*i + 1.
The right child is at index 2*i + 2.
The parent of a node at index i is at index (i-1)/2.
Takes array elements as input and displays the binary tree elements.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Array index out of bounds	Check the array size before accessing elements.
2	Incorrect left or right child	Use the correct formulas: 2*i + 1 and 2*i + 2.
3	Missing child nodes	Use -1 or another marker to represent empty nodes.

7B)

AIM: Write a python program to implement binary tree using an linked list

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class BinaryTree:
    def __init__(self):
        self.root = None

    def create_tree(self):
        self.root = Node('A')
        self.root.left = Node('B')
        self.root.right = Node('C')
        self.root.left.left = Node('D')
        self.root.left.right = Node('E')
```

```
        self.root.right.right = Node('F')
        print("Binary tree created successfully!")

    def preorder(self, node):
        if node is None:
            return
        print(node.data, end=" ")
        self.preorder(node.left)
        self.preorder(node.right)

    def inorder(self, node):
        if node is None:
            return
        self.inorder(node.left)
        print(node.data, end=" ")
        self.inorder(node.right)

    def postorder(self, node):
        if node is None:
            return
        self.postorder(node.left)
        self.postorder(node.right)
        print(node.data, end=" ")

    def level_order(self):
        if self.root is None:
            print("Tree is empty!")
            return

        queue = [self.root]

        while queue:
            node = queue.pop(0)
            print(node.data, end=" ")

            if node.left is not None:
                queue.append(node.left)

            if node.right is not None:
                queue.append(node.right)


tree = BinaryTree()

while True:
    print("\n\n--- BINARY TREE USING LINKED NODES ---")
```

```
print("1. Create Tree")
print("2. Preorder Traversal")
print("3. Inorder Traversal")
print("4. Postorder Traversal")
print("5. Level-order Traversal")
print("0. Exit")

while True:
    choice = int(input("Enter your choice: "))

    if choice == 1:
        tree.create_tree()

    elif choice == 2:
        if tree.root is None:
            print("Create the tree first!")
        else:
            print("Preorder: ", end="")
            tree.preorder(tree.root)
            print()

    elif choice == 3:
        if tree.root is None:
            print("Create the tree first!")
        else:
            print("Inorder: ", end="")
            tree.inorder(tree.root)
            print()

    elif choice == 4:
        if tree.root is None:
            print("Create the tree first!")
        else:
            print("Postorder: ", end="")
            tree.postorder(tree.root)
            print()

    elif choice == 5:
        print("Level-order: ", end="")
        tree.level_order()
        print()

    elif choice == 0:
        print("Exiting program...")
        break
```

```
    else:
        print("Invalid choice! Try again.")
```

OUTPUT:

```
======= RESTART: C:\Users\abdul\OneDrive\Documents\dsapy_lab\lab_7\7b.py =======


--- BINARY TREE USING LINKED NODES ---
1. Create Tree
2. Preorder Traversal
3. Inorder Traversal
4. Postorder Traversal
5. Level-order Traversal
0. Exit
Enter your choice: 1
Binary tree created successfully!
Enter your choice: 2
Preorder: A B D E C F
Enter your choice: 3
Inorder: D B E A C F
Enter your choice: 3
Inorder: D B E A C F
Enter your choice:
```

EXPLANATION:

Defines a binary tree using a linked list to store the elements.
Each node contains three fields: data, left pointer, and right pointer.
The root node represents the starting node of the binary tree.
The left pointer stores the address of the left child node.
The right pointer stores the address of the right child node.
Takes node values as input and displays the binary tree elements.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Null pointer error	Check whether the node pointer is NULL before accessing it.
2	Incorrect left or right child	Assign the new node to the correct left or right pointer.
3	Memory allocation failure	Check whether malloc() returns NULL.

LAB 8

8A)

AIM: Write a Python program to implement a Binary Search Tree (BST)

PROGRAM:

```
tree = []
def insert(value, i=0):
    global tree

    if i >= len(tree):
        tree.extend([None] * (i - len(tree) + 1))

    if tree[i] is None:
        tree[i] = value
    elif value < tree[i]:
        insert(value, 2 * i + 1)
    elif value > tree[i]:
        insert(value, 2 * i + 2)
    else:
        print("Duplicate values are not allowed.")


def search(value, i=0):
    if i >= len(tree) or tree[i] is None:
        return False

    if tree[i] == value:
        return True
    elif value < tree[i]:
        return search(value, 2 * i + 1)
    else:
        return search(value, 2 * i + 2)


def minimum(i):
    while 2 * i + 1 < len(tree) and tree[2 * i + 1] is not None:
        i = 2 * i + 1
    return i


def delete(value, i=0):
    if i >= len(tree) or tree[i] is None:
        return
```

```
    if value < tree[i]:
        delete(value, 2 * i + 1)
    elif value > tree[i]:
        delete(value, 2 * i + 2)
    else:
        left = 2 * i + 1
        right = 2 * i + 2

        has_left = left < len(tree) and tree[left] is not None
        has_right = right < len(tree) and tree[right] is not None

        if not has_left and not has_right:
            tree[i] = None

        elif not has_left:
            tree[i] = tree[right]
            delete_subtree(right, i)

        elif not has_right:
            tree[i] = tree[left]
            delete_subtree(left, i)

        else:
            j = minimum(right)
            tree[i] = tree[j]
            delete(tree[j], right)


def delete_subtree(child, parent):
    if child >= len(tree) or tree[child] is None:
        return

    value = tree[child]
    delete(value, child)

    remaining = collect_subtree(child)
    for v in remaining:
        insert(v)

def collect_subtree(i):
    if i >= len(tree) or tree[i] is None:
        return []
    value = tree[i]
    left = collect_subtree(2 * i + 1)
    right = collect_subtree(2 * i + 2)
    tree[i] = None
```

```
    return left + [value] + right


def inorder(i=0):
    if i < len(tree) and tree[i] is not None:
        inorder(2 * i + 1)
        print(tree[i], end=" ")
        inorder(2 * i + 2)


def preorder(i=0):
    if i < len(tree) and tree[i] is not None:
        print(tree[i], end=" ")
        preorder(2 * i + 1)
        preorder(2 * i + 2)


def postorder(i=0):
    if i < len(tree) and tree[i] is not None:
        postorder(2 * i + 1)
        postorder(2 * i + 2)
        print(tree[i], end=" ")


while True:
    print("\n\n1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Display Array")
    print("8. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        insert(value)

    elif choice == 2:
        value = int(input("Enter value to delete: "))
        if search(value):
            delete(value)
            print("Value deleted.")
        else:
```

```
            print("Value not found.")

    elif choice == 3:
        value = int(input("Enter value to search: "))
        print("Found" if search(value) else "Not found")

    elif choice == 4:
        inorder()
        print()

    elif choice == 5:
        preorder()
        print()

    elif choice == 6:
        postorder()
        print()

    elif choice == 7:
        print(tree)

    elif choice == 8:
        break

    else:
        print("Invalid choice.")
```

OUTPUT:

```
======= RESTART: C:/Users/abdul/OneDrive/Documents/dsapy_lab/lab_8/8a.py =======


1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Array
8. Exit
Enter your choice: 1
Enter value: 12
```

EXPLANATION:

Defines a Binary Search Tree (BST) using an array to store the elements.
Inserts elements according to the BST property: smaller values go to the left, and larger values go to the right.
Uses index 0 to store the root node.
For a node at index i, the left child is at index 2*i + 1 and the right child is at index 2*i + 2.
Performs insertion, deletion, searching, and traversal using a menu-driven program.
Displays the elements using inorder, preorder, and postorder traversal.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Array index out of bounds	Extend the array when a required index exceeds its size.
2	Incorrect insertion position	Compare values and use the correct left and right child indices.
3	Element not found during deletion	Search for the element before deleting it.

8B)

AIM: Write a Python program to implement a Binary Search Tree (BST) use Ll

PROGRAM:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert(root, value):
    if root is None:
        return Node(value)

    if value < root.data:
        root.left = insert(root.left, value)
    elif value > root.data:
        root.right = insert(root.right, value)
    else:print("Duplicate values are not allowed.")
```

```
    return root

def search(root, value):
    if root is None:
        return False

    if root.data == value:
        return True
    elif value < root.data:
        return search(root.left, value)
    else:
        return search(root.right, value)

def find_min(root):
    while root.left is not None:
        root = root.left
    return root

def delete(root, value):
    if root is None:
        return None

    if value < root.data:
        root.left = delete(root.left, value)

    elif value > root.data:
        root.right = delete(root.right, value)

    else:
        # No left child
        if root.left is None:
            return root.right

        # No right child
        if root.right is None:
            return root.left

        # Two children
        temp = find_min(root.right)
        root.data = temp.data
        root.right = delete(root.right, temp.data)

    return root

def inorder(root):
```

```
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)

def preorder(root):
    if root:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)

def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")

root = None

while True:
    print("\n\n1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        root = insert(root, value)

    elif choice == 2:
        value = int(input("Enter value to delete: "))
        if search(root, value):
            root = delete(root, value)
            print("Value deleted.")
        else:
            print("Value not found.")

    elif choice == 3:
        value = int(input("Enter value to search: "))
        if search(root, value):
            print("Value found.")
```

```
        else:
            print("Value not found.")

    elif choice == 4:
        inorder(root)
        print()

    elif choice == 5:
        preorder(root)
        print()

    elif choice == 6:
        postorder(root)
        print()

    elif choice == 7:
        break

    else:
        print("Invalid choice.")
```

OUTPUT:

```
======= RESTART: C:/Users/abdul/OneDrive/Documents/dsapy_lab/lab_8/8b.py =======


1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 1
Enter value: 12
```

EXPLANATION:

Defines a Binary Search Tree (BST) using a linked list structure.
Each node contains three fields: data, left, and right.
Inserts smaller values into the left subtree and larger values into the right subtree.
Uses a root pointer to represent the starting node of the tree.
Performs insertion, deletion, searching, and traversal using a menu-driven program.
Displays the elements using inorder, preorder, and postorder traversal.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Null pointer error	Check whether the node is None before accessing its data.
2	Incorrect insertion position	Compare the value with the current node and follow the correct pointer.
3	Incorrect deletion of a node	Handle leaf nodes, nodes with one child, and nodes with two children separately.

LAB 9

9A)

AIM: Write a Python program to implement a AVL tree using arrasy

PROGRAM:

```
keys = []
left = []
right = []
heights = []
root = -1

def new_node(key):
    i = len(keys)
    keys.append(key)
    left.append(-1)
    right.append(-1)
    heights.append(1)
    return i

def height(i):
    return 0 if i == -1 else heights[i]

def update(i):
    heights[i] = 1 + max(height(left[i]), height(right[i]))

def balance(i):
    return 0 if i == -1 else height(left[i]) - height(right[i])

def rotate_right(y):
    x = left[y]
    t = right[x]

    right[x] = y
    left[y] = t

    update(y)
    update(x)
    return x

def rotate_left(x):
    y = right[x]
    t = left[y]

    left[y] = x
```

```
    right[x] = t

    update(x)
    update(y)
    return y

def insert(node, key):
    if node == -1:
        return new_node(key)

    if key < keys[node]:
        left[node] = insert(left[node], key)
    elif key > keys[node]:
        right[node] = insert(right[node], key)
    else:
        return node

    update(node)
    bf = balance(node)

    if bf > 1 and key < keys[left[node]]:
        return rotate_right(node)

    if bf < -1 and key > keys[right[node]]:
        return rotate_left(node)

    if bf > 1 and key > keys[left[node]]:
        left[node] = rotate_left(left[node])
        return rotate_right(node)

    if bf < -1 and key < keys[right[node]]:
        right[node] = rotate_right(right[node])
        return rotate_left(node)

    return node

def min_node(node):
    while left[node] != -1:
        node = left[node]
    return node

def delete(node, key):
    if node == -1:
        return -1

    if key < keys[node]:
```

```
        left[node] = delete(left[node], key)
    elif key > keys[node]:
        right[node] = delete(right[node], key)
    else:
        if left[node] == -1:
            return right[node]
        if right[node] == -1:
            return left[node]

        temp = min_node(right[node])
        keys[node] = keys[temp]
        right[node] = delete(right[node], keys[temp])

    update(node)
    bf = balance(node)

    if bf > 1 and balance(left[node]) >= 0:
        return rotate_right(node)

    if bf > 1 and balance(left[node]) < 0:
        left[node] = rotate_left(left[node])
        return rotate_right(node)

    if bf < -1 and balance(right[node]) <= 0:
        return rotate_left(node)

    if bf < -1 and balance(right[node]) > 0:
        right[node] = rotate_right(right[node])
        return rotate_left(node)

    return node

def search(node, key):
    while node != -1:
        if keys[node] == key:
            return True
        if key < keys[node]:
            node = left[node]
        else:
            node = right[node]
    return False

def inorder(node):
    if node != -1:
        inorder(left[node])
        print(keys[node], end=" ")
```

```
        inorder(right[node])

def preorder(node):
    if node != -1:
        print(keys[node], end=" ")
        preorder(left[node])
        preorder(right[node])

def postorder(node):
    if node != -1:
        postorder(left[node])
        postorder(right[node])
        print(keys[node], end=" ")

while True:
    print("\n\n--- AVL TREE USING ARRAYS ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder")
    print("5. Preorder")
    print("6. Postorder")
    print("7. Exit")

    choice = input("Enter choice: ")

    if choice == '1':
        key = int(input("Enter value: "))
        root = insert(root, key)
        print("Node inserted.")

    elif choice == '2':
        key = int(input("Enter value: "))
        if search(root, key):
            root = delete(root, key)
            print("Node deleted.")
        else:
            print("Value not found.")

    elif choice == '3':
        key = int(input("Enter value: "))
        print("Value found." if search(root, key)
              else "Value not found.")

    elif choice == '4':
        inorder(root)
```

```
        print()

    elif choice == '5':
        preorder(root)
        print()

    elif choice == '6':
        postorder(root)
        print()

    elif choice == '7':
        break

    else:
        print("Invalid choice.")
```

OUTPUT:

```
======= RESTART: C:\Users\abdul\OneDrive\Documents\dsapy_lab\lab_9\9a.py =======


--- AVL TREE USING ARRAYS ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Exit
Enter choice: 1
Enter value: 12
Node inserted.
```

EXPLANATION:

Creates arrays to store node values, left-child indices, right-child indices, and node heights.
Inserts elements according to Binary Search Tree (BST) properties.
Calculates the height and balance factor of each node.
Performs LL, RR, LR, and RL rotations to maintain AVL tree balance.
Deletes nodes and rebalances the tree whenever necessary.
Searches for an element and displays whether it is found or not.
Performs inorder, preorder, and postorder traversals.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Incorrect child index	Verify left and right child indices are updated correctly during rotations.
2	Tree becomes unbalanced	Calculate the balance factor and apply the appropriate rotation.
3	Incorrect height calculation	Update node heights after insertion, deletion, and rotations.

9B)

AIM: Write a Python program to implement a AVL tree using linked list

PROGRAM:

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1

def height(root):
    return 0 if root is None else root.height

def balance(root):
    if root is None:
        return 0
    return height(root.left) - height(root.right)

def update(root):
    root.height = 1 + max(height(root.left), height(root.right))

def rotate_right(y):
    x = y.left
    t = x.right

    x.right = y
    y.left = t

    update(y)
    update(x)
```

```
    return x

def rotate_left(x):
    y = x.right
    t = y.left

    y.left = x
    x.right = t

    update(x)
    update(y)
    return y

def insert(root, key):
    if root is None:
        return Node(key)

    if key < root.key:
        root.left = insert(root.left, key)
    elif key > root.key:
        root.right = insert(root.right, key)
    else:
        return root

    update(root)
    bf = balance(root)

    if bf > 1 and key < root.left.key:
        return rotate_right(root)

    if bf < -1 and key > root.right.key:
        return rotate_left(root)

    if bf > 1 and key > root.left.key:
        root.left = rotate_left(root.left)
        return rotate_right(root)

    if bf < -1 and key < root.right.key:
        root.right = rotate_right(root.right)
        return rotate_left(root)

    return root

def min_node(root):
    while root.left is not None:
        root = root.left
```

```
    return root

def delete(root, key):
    if root is None:
        return None

    if key < root.key:
        root.left = delete(root.left, key)
    elif key > root.key:
        root.right = delete(root.right, key)
    else:
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left

        temp = min_node(root.right)
        root.key = temp.key
        root.right = delete(root.right, temp.key)

    update(root)
    bf = balance(root)

    if bf > 1 and balance(root.left) >= 0:
        return rotate_right(root)

    if bf > 1 and balance(root.left) < 0:
        root.left = rotate_left(root.left)
        return rotate_right(root)

    if bf < -1 and balance(root.right) <= 0:
        return rotate_left(root)

    if bf < -1 and balance(root.right) > 0:
        root.right = rotate_right(root.right)
        return rotate_left(root)

    return root

def search(root, key):
    if root is None or root.key == key:
        return root

    if key < root.key:
        return search(root.left, key)
```

```
    return search(root.right, key)

def inorder(root):
    if root:
        inorder(root.left)
        print(root.key, end=" ")
        inorder(root.right)

def preorder(root):
    if root:
        print(root.key, end=" ")
        preorder(root.left)
        preorder(root.right)

def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.key, end=" ")

root = None

while True:
    print("\n\n--- AVL TREE USING LINKED LIST ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder")
    print("5. Preorder")
    print("6. Postorder")
    print("7. Exit")

    choice = input("Enter choice: ")

    if choice == '1':
        key = int(input("Enter value: "))
        root = insert(root, key)
        print("Node inserted.")

    elif choice == '2':
        key = int(input("Enter value: "))
        if search(root, key):
            root = delete(root, key)
            print("Node deleted.")
        else:
            print("Value not found.")
```

```
    elif choice == '3':
        key = int(input("Enter value: "))
        if search(root, key):
            print("Value found.")
        else:
            print("Value not found.")

    elif choice == '4':
        inorder(root)
        print()

    elif choice == '5':
        preorder(root)
        print()

    elif choice == '6':
        postorder(root)
        print()

    elif choice == '7':
        print("Exiting program.")
        break

    else:
        print("Invalid choice.")
```

OUTPUT:

```
======= RESTART: C:\Users\abdul\OneDrive\Documents\dsapy_lab\lab_9\9b.py =======


--- AVL TREE USING LINKED LIST ---
1. Insert
2. Delete
3. Search
4. Inorder
5. Preorder
6. Postorder
7. Exit
Enter choice: 2
Enter value: 12
Value not found.
```

EXPLANATION:

Creates nodes containing a key, left-child reference, right-child reference, and height.
Inserts elements according to Binary Search Tree (BST) properties.
Calculates the height and balance factor of each node.
Performs LL, RR, LR, and RL rotations to maintain AVL tree balance.
Deletes nodes and restores balance using rotations.
Searches for an element by comparing the target with each node's key.
Performs inorder, preorder, and postorder traversals.
Displays whether the searched element is found or not.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	AttributeError while accessing a node	Check whether the node is None before accessing its attributes.
2	Tree becomes unbalanced	Calculate the balance factor and apply the correct rotation.
3	Incorrect height calculation	Update the heights of affected nodes after every rotation.

LAB 10

10A)

AIM: Write a Python program to implement Hashing using Separate Chaining

PROGRAM:

```
SIZE = 10
table = [[] for _ in range(SIZE)]

def insert(key):
    index = key % SIZE
    table[index].append(key)
    print("Inserted successfully")

def search(key):
    index = key % SIZE

    if key in table[index]:
        print("Element found")
    else:
        print("Element not found")

def display():
    for i in range(SIZE):
        print(i, ":", table[i])

while True:
    print("\n1.Insert 2.Search 3.Display 4.Exit")
    ch = int(input("Enter choice: "))

    if ch == 1:
        key = int(input("Enter key: "))
        insert(key)

    elif ch == 2:
        key = int(input("Enter key: "))
        search(key)

    elif ch == 3:
        display()

    elif ch == 4:
        break
```

```
    else:
        print("Invalid choice")
```

OUTPUT:

```
====== RESTART: C:\Users\abdul\OneDrive\Documents\dsapy_lab\lab_10\10a.py ======

1.Insert 2.Search 3.Display 4.Exit
Enter choice: 1
Enter key: 12
Inserted successfully

1.Insert 2.Search 3.Display 4.Exit
Enter choice:
```

EXPLANATION:

Creates a hash table using an array of lists to store elements.
Calculates the hash index using the hash function key % SIZE.
Inserts elements into the corresponding index of the hash table.
Handles collisions by storing multiple elements in the same list.
Searches for an element in the corresponding list and displays whether it is found or not.
Displays all hash table indices and their stored elements.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Incorrect hash index	Calculate the index using key % SIZE.
2	Element not inserted properly	Use append() to insert elements into the corresponding list.
3	Element not found during searching	Search within the correct index list.

10B)

AIM: Write a Python program to implement Hashing using Linear Probing

PROGRAM:

```
SIZE = 10
table = [-1] * SIZE

def insert(key):
    index = key % SIZE

    for i in range(SIZE):
        pos = (index + i) % SIZE

        if table[pos] == -1:
            table[pos] = key
            print("Inserted successfully")
            return

    print("Hash table is full")

def search(key):
    index = key % SIZE

    for i in range(SIZE):
        pos = (index + i) % SIZE

        if table[pos] == -1:
            break

        if table[pos] == key:
            print("Element found at index", pos)
            return

    print("Element not found")

def display():
    for i in range(SIZE):
        print(i, ":", table[i])

while True:
    print("\n1.Insert 2.Search 3.Display 4.Exit")
    ch = int(input("Enter choice: "))

    if ch == 1:
```

```
        key = int(input("Enter key: "))
        insert(key)

    elif ch == 2:
        key = int(input("Enter key: "))
        search(key)

    elif ch == 3:
        display()

    elif ch == 4:
        break

    else:
        print("Invalid choice")
```

OUTPUT:

```
====== RESTART: C:/Users/abdul/OneDrive/Documents/dsapy_lab/lab_10/10b.py ======

1.Insert 2.Search 3.Display 4.Exit
Enter choice: 2
Enter key: 12
Element not found

1.Insert 2.Search 3.Display 4.Exit
Enter choice:
```

EXPLANATION:

Creates an array to store elements in the hash table and initializes all positions with -1.
Calculates the hash index using the hash function key % SIZE.
Inserts elements according to the linear probing technique.
Handles collisions by checking the next consecutive position using (index + i) % SIZE.
Searches for an element using the same probing sequence and displays whether it is found or not.
Displays all hash table indices and their stored elements.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Incorrect hash index	Calculate the index using key % SIZE.

2	Collision not handled properly	Check the next available position using linear probing.
3	Element not found during searching	Use the same probing sequence used during insertion.

10C)

AIM: Write a Python program to implement Hashing using Quadratic Probing

PROGRAM:

```
SIZE = 10
table = [-1] * SIZE

def insert(key):
    index = key % SIZE

    for i in range(SIZE):
        pos = (index + i * i) % SIZE

        if table[pos] == -1:
            table[pos] = key
            print("Inserted successfully")
            return

    print("Insertion failed")

def search(key):
    index = key % SIZE

    for i in range(SIZE):
        pos = (index + i * i) % SIZE

        if table[pos] == -1:
            break

        if table[pos] == key:
            print("Element found at index", pos)
            return

    print("Element not found")

def display():
```

```
    for i in range(SIZE):
        print(i, ":", table[i])

while True:
    print("\n1.Insert 2.Search 3.Display 4.Exit")
    ch = int(input("Enter choice: "))

    if ch == 1:
        key = int(input("Enter key: "))
        insert(key)

    elif ch == 2:
        key = int(input("Enter key: "))
        search(key)

    elif ch == 3:
        display()

    elif ch == 4:
        break

    else:
        print("Invalid choice")
```

OUTPUT:

```
====== RESTART: C:/Users/abdul/OneDrive/Documents/dsapy_lab/lab_10/10c.py ======

1.Insert 2.Search 3.Display 4.Exit
Enter choice: 3
0 : -1
1 : -1
2 : -1
3 : -1
4 : -1
5 : -1
6 : -1
7 : -1
8 : -1
9 : -1

1.Insert 2.Search 3.Display 4.Exit
Enter choice:
```

EXPLANATION:

Creates an array to store elements in the hash table and initializes all positions with -1.
Calculates the hash index using the hash function key % SIZE.
Inserts elements according to the quadratic probing technique.
Handles collisions by checking positions using the formula (index + i*i) % SIZE.
Searches for an element using the same quadratic probing sequence and displays whether it is found or not.
Displays all hash table indices and their stored elements.

ERROR TABLE:

S.No	Error Obtained	Rectification
1	Incorrect child index	Calculate the index using key % SIZE..
2	Collision not resolved properly	Apply quadratic probing using index + i*i.
3	Element not found during searching	Use the same quadratic probing sequence used during insertion.